

Operazioni su file e descrittori

ESEMPIO LETTURA E SCRITTURA SU FILE:

```
#define DEFAULT_BLOCK_SIZE 128;

static inline void performCopyBetweenDescriptors(int src_fd, int dest_fd, int block_size)
{
    char* buf = malloc(block_size);
    //LETTURA SU UN FD
    while (1) {
        //READ
        int read_bytes = 0; // index for writing into the buffer
        int bytes_left = block_size; // number of bytes to (possibly) read
        while (bytes_left > 0) {
            int ret = read(src_fd, buf+read_bytes, bytes_left); //nota che legge da src, e
            //scrive nel buffer nella giusta posizione
            if (ret==0) break;
            if (ret == -1) {
                if (errno == EINTR) continue; //interrotto prima della lettura di un byte
                else exit(EXIT_FAILURE); //interrotto per qualche errore
            }
            bytes_left-=ret;//può capitare di leggere meno bytes del block_size. allora
            //aggiorno le variabili e continuo
            read_bytes+=ret;//finchè non lo leggo tutto
        }
        if (read_bytes == 0) break;

        //SCITTURA SU UN FD
        int written_bytes = 0; // index for reading from the buffer
        bytes_left = read_bytes; // number of bytes to write

        while (bytes_left > 0) {
            int ret = write(dest_fd, buf+written_bytes, bytes_left);
            if (ret==-1){
                if(errno == EINTR) continue;
                else exit(EXIT_FAILURE);
            }
            bytes_left-=ret;
            written_bytes+=ret;
        }
    }

    free(buf);
}

int main(int argc, char* argv[]) {
    int block_size, src_fd, dest_fd;

    if (argc == 4) {
        block_size = atoi(argv[3]);
    } else {
        block_size = DEFAULT_BLOCK_SIZE;
    }
}
```

```

}

if (argc < 3 || argc > 4)
    handle_error("Syntax: <source_file> <dest_file> [<block_size>]\n");
if (block_size <= 0)
    handle_error("Blocksize must be positive");

// create descriptors for source and destination files
src_fd = open(argv[1], O_RDONLY);
if (src_fd < 0) handle_error("Could not open source file");

// for simplicity we use rw-r--r-- permissions for the destination file
dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_EXCL, 0644);
if (dest_fd < 0){
    if(errno == EEXIST) {
        fprintf(stderr, "WARNING: file %s already exists, I will overwrite it!\n",
argv[2]);
        dest_fd = open(argv[2], O_WRONLY | O_CREAT, 0644);
    }else
        handle_error("Could not create destination file");
}

// use a helper method to actually perform the copy
performCopyBetweenDescriptors(src_fd, dest_fd, block_size);

// close the descriptors
int ret = close(src_fd);
if (ret < 0) handle_error("Could not close source file");
ret = close(dest_fd);
if (ret < 0) handle_error("Could not close destination file");

exit(EXIT_SUCCESS);
}

```

Semafori

ESEMPIO DI APPLICAZIONE SEMAFORI NORMALI

Per le named è la stessa cosa ma si dovrebbe fare una `sem_open` speciale

```
//Nel caso 1-1 facciamo un solo semaforo per l'accesso. l'ordine di semafori
//è stato sem_wait(sem_access) - sem_post(sem_produced) - sem_wait(sem_produced) -
sem_post(sem_access)
//come se fosse un unico flusso. questo non sarà possibile successivamente

//Nel caso 1-M. faccio due sem_access, sem_access1 e sem_access2, per fare in modo che
//non ci sia busy waiting tra i consumatori e i produttori
//ordine sem_wait(sem_access1) - sem_post(sem_produced) - sem_wait(sem_produced) -
sem_wait(sem_access2) -
//- sem_post(sem_access2) - sem_post(sem_access1). Nota come access1 sta all'inizio ed
alla fine

//Caso N-1: Semafori. ordine: sem_wait(sem_limit) - sem_wait(sem_access1) -
sem_post(sem_access1) - sem_post(sem_produced) - sem_wait(sem_produced) -
sem_post(sem_limit)
//sem_limit è un semaforo che mi dice quanti posti liberi ci sono nel buffer

//CASO GLOBALE!! N consumer e M producer

#define BUFFER_SIZE          128
#define INITIAL_DEPOSIT      0
#define MAX_TRANSACTION      1000
#define NUM_CONSUMERS        2
#define NUM_PRODUCERS        4
#define PRNG_SEED            0

#define NUM_OPERATIONS        400
#define OPS_PER_CONSUMER      (NUM_OPERATIONS/NUM_CONSUMERS)
#define OPS_PER_PRODUCER      (NUM_OPERATIONS/NUM_PRODUCERS)

// struct used to specify arguments for a thread
typedef struct {
    int threadId;
    int numOps;
} thread_args_t;

// shared data
int transactions[BUFFER_SIZE];
int deposit = INITIAL_DEPOSIT;
int read_index, write_index;

//Semafori
sem_t sem_access1, sem_access2, sem_produced, sem_limit;

// producer thread
void* performTransactions(void* x) {
    thread_args_t* args = (thread_args_t*)x;
```

```

printf("Starting producer thread %d\n", args->threadId);

while (args->numOps > 0) {
    if (sem_wait(&sem_limit) == -1){
        fprintf(stderr, "Errore nella sem_wait del semaforo sem_limit nel
performTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_wait(&sem_access1) == -1){
        fprintf(stderr, "Errore nella sem_wait del semaforo sem_access nel
performTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    // produce the item
    int currentTransaction = performRandomTransaction();

    // write the item and update write_index accordingly
    transactions[write_index] = currentTransaction;
    write_index = (write_index + 1) % BUFFER_SIZE;

    args->numOps--;
    //printf("P %d\n", args->numOps);

    if (sem_post(&sem_access1) == -1){
        fprintf(stderr, "Errore nella sem_post del semaforo sem_access nel
performTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_post(&sem_produced) == -1){
        fprintf(stderr, "Errore nella sem_post del semaforo sem_produced nel
performTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
}
free(args);
pthread_exit(NULL);
}

void* processTransactions(void* x) {
    thread_args_t* args = (thread_args_t*)x;

    printf("Starting consumer thread %d\n", args->threadId);

    while (args->numOps > 0) {
        if (sem_wait(&sem_produced) == -1){
            fprintf(stderr, "Errore nella sem_wait del semaforo sem_produced nel
processTransaction... error %s\n", errno);
            exit(EXIT_FAILURE);
        }
        if (sem_wait(&sem_access2) == -1){
            fprintf(stderr, "Errore nella sem_wait del semaforo sem_access nel
processTransaction... error %s\n", errno);
            exit(EXIT_FAILURE);
        }
    }
}

```

```

    }
    // consume the item and update (shared) variable deposit
    deposit += transactions[read_index];
    read_index = (read_index + 1) % BUFFER_SIZE;
    if (read_index % 100 == 0)
        printf("After the last 100 transactions balance is now %d.\n", deposit);

    args->numOps--;
    //printf("C %d\n", args->numOps);

    if (sem_post(&sem_access2) == -1){
        fprintf(stderr, "Errore nella sem_post del semaforo sem_access nel
processTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_post(&sem_limit) == -1){
        fprintf(stderr, "Errore nella sem_post del semaforo sem_limit nel
processTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
}

free(args);
pthread_exit(NULL);
}

int main(int argc, char* argv[]) {
    // initialize read and write indexes
    read_index = 0;
    write_index = 0;

    int ret;
    pthread_t producer[NUM_PRODUCERS], consumer[NUM_CONSUMERS];

    //inizializzo semafori
    if (sem_init(&sem_access1, 0, 1) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_access... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_init(&sem_access2, 0, 1) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_access... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_init(&sem_produced, 0, 0) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_produced... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_init(&sem_limit, 0, BUFFER_SIZE) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_limit... error
%s\n", errno);

```

```

    exit(EXIT_FAILURE);
}

int i;
//invia i producers
for (i=0; i<NUM_PRODUCERS; ++i) {
    thread_args_t* arg = malloc(sizeof(thread_args_t));
    arg->threadId = i;
    arg->numOps = OPS_PER_PRODUCER;

    ret = pthread_create(&producer[i], NULL, performTransactions, arg);
    if (ret != 0) { fprintf(stderr, "Error %d in pthread_create\n", ret);
exit(EXIT_FAILURE); }
}

int j;
//invia i consumers
for (j=0; j<NUM_CONSUMERS; ++j) {
    thread_args_t* arg = malloc(sizeof(thread_args_t));
    arg->threadId = j;
    arg->numOps = OPS_PER_CONSUMER;

    ret = pthread_create(&consumer[j], NULL, processTransactions, arg);
    if (ret != 0) { fprintf(stderr, "Error %d in pthread_create\n", ret);
exit(EXIT_FAILURE); }
}

// join on threads
//recupera i producers
for (i=0; i<NUM_PRODUCERS; ++i) {
    ret = pthread_join(producer[i], NULL);
    if (ret != 0) { fprintf(stderr, "Error %d in pthread_join\n", ret);
exit(EXIT_FAILURE); }
}
//recupera i consumers
for (j=0; j<NUM_CONSUMERS; ++j) {
    ret = pthread_join(consumer[j], NULL);
    if (ret != 0) { fprintf(stderr, "Error %d in pthread_join\n", ret);
exit(EXIT_FAILURE); }
}

printf("Final value for deposit: %d\n", deposit);

//distruggo i semafori
if (sem_destroy(&sem_access1) == -1){
    fprintf(stderr, "Errore nella distruzione del semaforo sem_access... error %s\n",
errno);
    exit(EXIT_FAILURE);
}
if (sem_destroy(&sem_access2) == -1){
    fprintf(stderr, "Errore nella distruzione del semaforo sem_access... error %s\n",
errno);
    exit(EXIT_FAILURE);
}

```

```
    if (sem_destroy(&sem_produced) == -1){
        fprintf(stderr, "Errore nella distruzione del semaforo sem_produced... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_destroy(&sem_limit) == -1){
        fprintf(stderr, "Errore nella distruzione del semaforo sem_limit... error %s\n",
errno);
        exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}
```

Shared Memory

PRODUCER CONSUMER CON SEMAFORI

producer.c

```
struct shared_memory {
    int buf [BUFFER_SIZE];
    int read_index;
    int write_index;
};

struct shared_memory *myshm_ptr;
int fd_shm;

sem_t *sem_empty, *sem_filled, *sem_cs;

void initMemory() {
    shm_unlink(SH_MEM_NAME);

    if((fd_shm = shm_open(SH_MEM_NAME, O_CREAT | O_EXCL , 0666)) < 0){
        handle_error("producer: initMemory shm_open error");
    }

    if(ftruncate (fd_shm, sizeof(struct shared_memory))){
        handle_error("producer: initMemory ftruncate error");
    }

    if((myshm_ptr = mmap (NULL, sizeof(struct shared_memory), PROT_READ | PROT_WRITE, MAP_SHARED,
fd_shm, 0)) == MAP_FAILED){
        handle_error("consumer: initMemory mmap error");
    }
    memset(myshm_ptr, 0, sizeof(struct shared_memory));
}

void closeMemory() {
    if(munmap(myshm_ptr, sizeof(struct shared_memory))){
        handle_error("producer: closeMemory munmap error");
    }
    if(close(fd_shm)){
        handle_error("producer: closeMemory close (fd) error");
    }
    if(shm_unlink(SH_MEM_NAME)){
```



```

    handle_error("producer: closeMemory shm_unlink error");
}
}

void initSemaphores() {}

void closeSemaphores() {}

static inline int performRandomTransaction() {}

void produce(int id, int numOps) {
    int localSum = 0;
    while (numOps > 0) {
        // producer, just do your thing!
        int value = performRandomTransaction();

        int ret = sem_wait(sem_empty);
        if (ret) handle_error("sem_wait empty\n");

        ret = sem_wait(sem_cs);
        if (ret) handle_error("sem_wait cs");

        myshm_ptr->buf[myshm_ptr->write_index] = value;
        myshm_ptr->write_index = (myshm_ptr->write_index + 1) % BUFFER_SIZE;

        ret = sem_post(sem_cs);
        if (ret) handle_error("sem_post cs");

        ret = sem_post(sem_filled);
        if (ret) handle_error("sem_post filled");

        localSum += value;
        numOps--;
    }
    printf("Producer %d ended. Local sum is %d\n", id, localSum);
}

int main(int argc, char** argv) {
    srand(PRNG_SEED);
    initSemaphores();
    initMemory(); //definita sopra

    int i, ret;
    for (i=0; i<NUM_PRODUCERS; ++i) {
        pid_t pid = fork();

```

```

    if (pid == -1) {
        handle_error("fork");
    } else if (pid == 0) {
        produce(i, OPS_PER_PRODUCER);
        _exit(EXIT_SUCCESS);
    }
}

for (i=0; i<NUM_PRODUCERS; ++i) {
    int status;
    ret = wait(&status);
    if (ret == -1) handle_error("wait");
    if (WEXITSTATUS(status)) handle_error_en(WEXITSTATUS(status), "child crashed");
}

printf("Producers have terminated. Exiting...\n");
closeSemaphores();
closeMemory(); //definita sopra
exit(EXIT_SUCCESS);
}

```

consumer.c

```

struct shared_memory {
    int buf [BUFFER_SIZE];
    int read_index;
    int write_index;
};

struct shared_memory *myshm_ptr;
int fd_shm;

sem_t *sem_empty, *sem_filled, *sem_cs;

void openMemory() {
    if ((fd_shm = shm_open(SH_MEM_NAME, O_RDWR, 0660)) == -1) handle_error("errore apertura
shared memory");
    if((myshm_ptr = mmap(NULL, sizeof(struct shared_memory), PROT_READ | PROT_WRITE, MAP_SHARED,
fd_shm, 0)) == MAP_FAILED){
        handle_error("consumer: openMemory mmap error");
    }
}
//A differenza del producer, non fa la ftruncate ed il memset

```

```

}

void closeMemory() {
    if(munmap(myshm_ptr, sizeof(struct shared_memory))){
        handle_error("consumer: closeMemory munmap error");
    }
    close(fd_shm);
    //non fa la shm_unlink per rendere accessibile la memoria ad altri consumer
}

void openSemaphores() {}

void closeAndDestroySemaphores() {}

void consume (int id, int numOps) {
    int localSum = 0;
    while (numOps > 0) {
        int ret = sem_wait(sem_filled);
        if (ret) handle_error("sem_wait filled");

        ret = sem_wait(sem_cs);
        if (ret) handle_error("sem_wait cs");

        int value = myshm_ptr->buf[myshm_ptr->read_index];
        myshm_ptr->read_index = (myshm_ptr->read_index + 1) % BUFFER_SIZE;

        ret = sem_post(sem_cs);
        if (ret) handle_error("sem_post cs");

        ret = sem_post(sem_empty);
        if (ret) handle_error("sem_post empty");

        localSum += value;
        numOps--;
    }
    printf("Consumer %d ended. Local sum is %d\n", id, localSum);
}

int main (int argc, char** argv) {
    openSemaphores();
    openMemory(); //definita sopra
    int i;
    for (i=0; i<NUM_CONSUMERS; ++i) {
        pid_t pid = fork();

```

```
if (pid == -1) {
    handle_error("fork");
} else if (pid == 0) {
    consume(i, OPS_PER_CONSUMER);
    _exit(EXIT_SUCCESS);
}
}

for (i=0; i<NUM_CONSUMERS; ++i) {
    int status;
    wait(&status);
    if (WEXITSTATUS(status)) handle_error("child crashed");
}

printf("Consumers have terminated. Exiting...\n");
closeAndDestroySemaphores();
closeMemory(); //definita sopra
exit(EXIT_SUCCESS); }
```

PIPE e FIFO

ESEMPIO DI UTILIZZO PIPE, LE FIFO SI USANO CON I PROCESSI

```
int pipefd[2];

int write_to_pipe(int fd, const void *data, size_t data_len) {
    //SCRITTURA IDENTICA COME SU FILE
    //USANDO LE PIPE

    int written_bytes = 0;
    int bytes_left = data_len;
    while (bytes_left > 0) {
        int ret = write(fd, data + written_bytes, bytes_left);
        if (ret == -1) {
            if (errno == EINTR) continue;
            else exit(EXIT_FAILURE);
        }
        bytes_left -= ret;
        written_bytes += ret;
    }
    //if (written_bytes == data_len) printf("sono stati scritti tutti i %d bytes",
written_bytes);
    return written_bytes;
}

int read_from_pipe(int fd, void *data, size_t data_len) { //LETTURA IDENTICA COME SU FILE
    //USANDO LE PIPE

    int read_bytes = 0;
    int bytes_left = data_len;
    while (bytes_left > 0) {
        if (bytes_left > 0) {
            int ret = read(fd, data + read_bytes, bytes_left);
            if (ret == -1) {
                if (errno == EINTR) continue;
                exit(EXIT_FAILURE);
            }
            if (ret == 0) break;

            bytes_left -= ret;
            read_bytes += ret;
        }
    }
    return read_bytes;
}

void reader(int reader_id, sem_t* read_mutex) {
    int data[MSG_ELEMS];
    int i, ret;
    printf("[READER_%d] processo reader creato.\n", reader_id);

    if (close(pipefd[1])) handle_error("errore chiusura");
```

```

    for (i = 0; i < MSG_COUNT/READERS_COUNT; i++) {

        ret = sem_wait(read_mutex);
        if(ret) handle_error("error waiting on read mutex");

FUNZIONE SOPRA-→ read_from_pipe(pipefd[0], data, sizeof(data)); // sizeof(data) ==
MSG_ELEMS * sizeof(int)

        ret = sem_post(read_mutex);
        if(ret) handle_error("error posting on read mutex");

        printf("[CHILD_%d] Letto msg #%d con valore %d\n", reader_id, i, data[0]);
        if (!is_msg_ok(data, MSG_ELEMS))
            printf("corrupted message!!!\n");
    }

    ret = sem_close(read_mutex);
    if(ret) handle_error("error closing read mutex");

    ret = close(pipefd[0]);
    if(ret) handle_error("error closing pipe");
}

void writer(int writer_id, sem_t* write_mutex) {
    int data[MSG_ELEMS];
    int i,ret;
    printf("[WRITER_%d] processo writer creato.\n", writer_id);

    if (close(pipefd[0])) handle_error("errore chiusura file descriptor lettura");

    for (i = 0 ; i < MSG_COUNT/WRITERS_COUNT; i++) {
        create_msg(data, MSG_ELEMS, i);
        ret = sem_wait(write_mutex);
        if(ret) handle_error("error waiting on write mutex");

        write_to_pipe(pipefd[1], data, sizeof(data)); // sizeof(data) == MSG_ELEMS *
sizeof(int)

        ret = sem_post(write_mutex);
        if(ret) handle_error("error posting on write mutex");

        printf("[WRITER_%d] Inviato il msg #%d\n", writer_id, i);
    }

    ret = sem_close(write_mutex);
    if(ret) handle_error("error closing write mutex");

    if (close(pipefd[1])) handle_error("errore chiusura file descriptor scrittura");
}

int main(int argc, char* argv[]) {

```

```

int ret,i;
pid_t pid;

sem_unlink(READ_MUTEX);
sem_t *read_mutex = sem_open(READ_MUTEX, O_CREAT|O_EXCL, 0600, 1);
if(read_mutex == SEM_FAILED) handle_error("Error Creating Read Mutex");

sem_unlink(WRITE_MUTEX);
sem_t *write_mutex = sem_open(WRITE_MUTEX, O_CREAT|O_EXCL, 0600, 1);
if(write_mutex == SEM_FAILED) handle_error("Error Creating Write Mutex");

if (pipe(pipefd)==-1) handle_error("errore apertura pipe!");

for (i = 0; i < READERS_COUNT; i++) {
    pid = fork();
    if (pid == -1) handle_error("Error creating reader");
    if (pid == 0) {
        reader(i,read_mutex);
        _exit(0);
    }
}

ret = sem_close(read_mutex);
if(ret) handle_error("Error closing read mutex");

for (i = 0; i < WRITERS_COUNT; i++) {
    pid = fork();
    if (pid == -1) handle_error("error creating reader");
    if (pid == 0) {
        writer(i,write_mutex);
        _exit(0);
    }
}

ret = sem_close(write_mutex);
if(ret) handle_error("Error closing write mutex");

// shutdown phase
for (i = 0 ; i < READERS_COUNT + WRITERS_COUNT; i++) {
    int status;
    ret = wait(&status);
    if(ret == 1) handle_error("error waiting for a child to terminate");
    if (WEXITSTATUS(status)) handle_error("child process died unexpectedly");
}

printf("[PARENT] processi figlio terminati.\n");

ret = sem_unlink(READ_MUTEX);
if(ret) handle_error("error removing read mutex");

ret = sem_unlink(WRITE_MUTEX);
if(ret) handle_error("error removing write mutex");

return 0; }

```

Socket e Network

ESEMPIO DI APPLICAZIONE

SERVER

```
int main(int argc, char* argv[]) {
    int ret;
    int socket_desc, client_desc;
    // some fields are required to be filled with 0
    struct sockaddr_in server_addr = {0}, client_addr = {0};    //STRUCT PER IL SERVER e
CLIENT

    int sockaddr_len = sizeof(struct sockaddr_in); // we will reuse it for accept()

    //SOCK_STREAM è TCP mentre SOCK_DGRAM è UDP
    if ( (socket_desc = socket(AF_INET, SOCK_STREAM, 0) ) == -1 )    //APERTURA SOCKET
handle_error("errore creazione socket server_addr");

    server_addr.sin_addr.s_addr = INADDR_ANY;    //compilo la struct del server,
server_addr.sin_family = AF_INET;    //non serve compilare il client
server_addr.sin_port = htons(SERVER_PORT);
    if (bind(socket_desc, (struct sockaddr*)&server_addr, sockaddr_len))==-1)    //BIND
        handle_error("errore binding server addr");
    ret = listen(socket_desc, MAX_CONN_QUEUE);    //ASCOLTO
    if (ret < 0)
        handle_error("Cannot listen on socket");
    // loop to handle incoming connections (sequentially)
    while (1) { //ACCETTO LA RICHIESTA DI CONNESSIONE DAL CLIENT
        client_desc = accept(socket_desc, (struct sockaddr*)
            &client_addr,(socklen_t*)&sockaddr_len);
        if (client_desc < 0) handle_error("Cannot open socket for incoming
            connection");
        connection_handler(client_desc); //è una funzione che ci creiamo noi per
            gestire l'invio e la ricezione
    }
    exit(EXIT_SUCCESS); // this will never be executed
}

// Method for processing incoming requests. The method takes as argument
// the socket descriptor for the incoming connection.
void* connection_handler(int socket_desc) {
    int ret, recv_bytes, bytes_sent;

    char buf[1024];
    size_t buf_len = sizeof(buf);
    int msg_len;
    memset(buf,0,buf_len);

    char* quit_command = SERVER_COMMAND;
    size_t quit_command_len = strlen(quit_command);

    // send welcome message
```



```

sprintf(buf, "Hi! I'm an echo server. I will send you back whatever"
        " you send me. I will stop if you send me %s", quit_command);
msg_len = strlen(buf);

bytes_sent=0;
while ( bytes_sent < msg_len) {
ret = send(socket_desc, buf + bytes_sent, msg_len - bytes_sent, 0); //INVIO //il numero di
byte da //inviare. GUARDA A FINE PAGINA!

    if (ret == -1 && errno == EINTR) continue;
    if (ret == -1) handle_error("Cannot write to the socket");
    bytes_sent += ret;
}

if (DEBUG) fprintf(stderr, "Welcome message <<%s>> has been sent\n",buf);
// echo loop
while (1) {
    recv_bytes = 0;
    do {
        ret = recv(socket_desc, buf + recv_bytes, 1, 0); //RICEVO
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from the socket");
        if (ret == 0) break;
    } while ( buf[recv_bytes++] != '\n' );

    if (DEBUG) fprintf(stderr, "Received command of %d bytes...\n",recv_bytes);
    // check if either I have just been told to quit...
    if (recv_bytes == quit_command_len && !memcmp(buf, quit_command,
quit_command_len)){
        break;
    }
    // ...or I have to send the message back
    bytes_sent=0;
    while ( bytes_sent < recv_bytes) {
        ret = send(socket_desc, buf + bytes_sent, recv_bytes - bytes_sent,
0); //INVIO conoscendo
        if (ret == -1 && errno == EINTR) continue; //il numero di
byte
        if (ret == -1) handle_error("Cannot write to the socket"); //da
inviare
        bytes_sent += ret;
    }
}
ret = close(socket_desc); //CHIUDO
if (ret < 0) handle_error("Cannot close socket for incoming connection");
return NULL;
}

```

GUARDA QUI PER LA LETTURA DI UN NUMERO DI BYTE PER VOLTA FINO AL CARATTERE \n

```
//LEGGE UN CARATTERE PER VOLTA
do{
    ret = recv(socket_desc, buf+bytes_read, 1, 0);
    if (ret ==-1){
        if (errno == EINTR) continue;
        handle_error("errore nella read. probabile chiusura dei stream");
    }
    if (ret==0) break;
}while (buf[bytes_read++]!='\n'); //ATTENZIONE! NOTA IL buf[bytes_read++] CON IL
++ DOPO! DEVE PRIMA LEGGERE IL buf[bytes_read] e poi incrementare altrimenti non troverà
mai il carattere \n
```

PER PATRIZIO CHE ODIA IL DO-WHILE

```
bytes_read = 0;
while(buf[bytes_read - 1] != '\n'){
    ret = recv(socket_desc, buf + bytes_read, 1, 0);
    if(ret == -1 && errno != EINTR){
        handle_error("[server] connection_handler recv error\n");
    }
    bytes_read += ret;
}
```

PATTERN PER RENDERE IL SERVER MULTIPROCESS OPPURE MULTITHREAD

// SE IL SERVER È` MULTIPROCESS, ALLORA BISOGNA FARE QUESTO CICLO IN FASE DI ACCETTAZIONE

//ithread hanno un impatto minore alla CPU rispetto al multiprocess che ha un overhead più alto

```
while (1) {
    pthread_t thread;
    struct sockaddr_in* client_addr = calloc(1, sizeof(struct sockaddr_in)); //lo crea per ogni thread

    client_desc = accept(socket_desc, (struct sockaddr*) client_addr, (socklen_t*) &sockaddr_len);
    if (client_desc == -1 && errno == EINTR) continue; // check per segnali di interruzione
    if (client_desc == -1) handle_error("Impossibile aprire la socket per connessioni in ingre

    handler_args_t* arg = calloc(1, sizeof(handler_args_t));
    arg->socket_desc = client_desc;
    arg->client_addr = client_addr;
    arg->client_id = client_id;
    ret = pthread_create(&thread, NULL, connection_handler, arg);
    if(ret) handle_error_en(ret,"Errore nella creazione del thread");

    ret = pthread_detach(thread);
    if(ret) handle_error_en(ret,"Errore nel detach del thread");

    // incremento il client ID dopo ogni connessione accettata
    client_id++;
}
```

PROSSIMA PAGINA SERVER MULTithread

```
// SE IL SERVER È MULTIPROCESS, ALLORA BISOGNA FARE QUESTO CICLO IN FASE DI ACCETTAZIONE
while (1){
    int client = accept(server, .....);
    <gestione errori>
    pid_t pid = fork();
    if (pid == -1){
        <gestione errori>
    } else if (pid == 0){
        close(server);
        <elaborazione connessione client>
        _exit(0);
    } else{
        close(client);
    }
}
```

CLIENT

```
int main(int argc, char* argv[]) {
    int ret, bytes_sent, recv_bytes;

    // variables for handling a socket
    int socket_desc;
    struct sockaddr_in server_addr = {0} //CREO STRUCT SERVER A CUI CONNETTERSI
                                         //porta TCP con SOCK_STREAM
    socket_desc = socket(AF_INET, SOCK_STREAM, 0); //APRO LA SOCKET DEL CLIENT,
                                                  //Usare SOCK_DGRAM per UDP

    if (socket_desc < 0)
        handle_error("Could not create socket");

    // inet_addr importante per il client, poichè
    server_addr.sin_addr.s_addr = inet_addr(SERVER_ADDRESS); //il server accetta tutte
                                                             //le interface ma il
    server_addr.sin_family = AF_INET; //client deve convertire la stringa in un IP
    server_addr.sin_port = htons(SERVER_PORT); // don't forget about
                                                //network byte order!

    //MI CONNETTO AL SERVER, BLOCCANTE FICNHÈ NON MI ACCETTA IL SERVER
    ret = connect(socket_desc, (struct sockaddr*) &server_addr,
                  sizeof(struct sockaddr_in));

    if (ret < 0)
        handle_error("Could not create connection");
    char buf[1024];
    size_t buf_len = sizeof(buf);
    int msg_len;
    memset(buf, 0, buf_len);
    recv_bytes = 0;
    do {
        //Se UDP recvfrom(socket_desc, buf, buf_len, 0, NULL, NULL)
        ret = recv(socket_desc, buf + recv_bytes, buf_len - recv_bytes, 0); //RICEVO MSG
                                                                              SERVER

        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from the socket");
        if (ret == 0) break;
    } while (ret > 0);
}
```

```

    recv_bytes += ret;
} while ( buf[recv_bytes-1] != '\n' ); //il -1 è importante perché sennò considero il
carattere di fine stringa //in C, cioè \0, e non troverà mai il \n che cerchiamo
// main loop
while (1) {
    char* quit_command = SERVER_COMMAND;
    size_t quit_command_len = strlen(quit_command);

    printf("Insert your message: ");

    /* Read a line from stdin
    *
    * fgets() reads up to sizeof(buf)-1 bytes and on success
    * returns the first argument passed to it. */
    if (fgets(buf, sizeof(buf), stdin) != (char*)buf) {
        fprintf(stderr, "Error while reading from stdin, exiting...\n");
        exit(EXIT_FAILURE);
    }

    msg_len = strlen(buf);
    // buf[--msg_len] = '\0'; // remove '\n' from the end of the message

    bytes_sent=0;
    while ( bytes_sent < msg_len) {
        //se è UDP, sendto(socket_desc, buf, msg_len, 0, (struct sockaddr*) &server_addr,
        //Sizeof(struct sockaddr_in))
        ret = send(socket_desc, buf + bytes_sent, msg_len - bytes_sent, 0);
        //INVIO AL SERVER

        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot write to the socket");
        bytes_sent += ret;
    }

    if (msg_len == quit_command_len && !memcmp(buf, quit_command, quit_command_len)){
        if (DEBUG) fprintf(stderr, "Sent QUIT command ...\n");
        break;
    }

    recv_bytes = 0;
    do {
        //se UDP recvfrom(socket_desc, buf, buf_len, 0, NULL, NULL)
        ret = recv(socket_desc, buf + recv_bytes, buf_len - recv_bytes, 0);
        //RICEVO (opzionale)

        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from the socket");
        if (ret == 0) break;
        recv_bytes += ret;
    } while ( buf[recv_bytes-1] != '\n' );
}
ret = close(socket_desc); //CHIUDO SOCKET CLIENT
if (ret < 0) handle_error("Cannot close the socket"); exit(EXIT_SUCCESS);
}

```

Dijkstra

Initially

```
/* global info */
boolean interested[N] = {false, ..., false};
boolean passed[N] = {false, ..., false};
int k = <any> ; // k ∈ {0, 1, ..., N-1}
/* local info */
int i = <entity ID>; // i ∈ {0, 1, ..., N-1}
```

repeat

```
1 <Non Critical Section>;
2 interested[i] = true;
3 while (k != i) {
4     passed[i] = false;
5     if (!interested[k]) then k = i;
6 }
7 passed[i] = true;
8 for j in 0 ... N-1 except i do
9     if (passed[j]) then goto 3;
10 <critical section>;
11 passed[i] = false; interested[i] = false;
```

forever

Possibili domande:

- cosa cambia se si nega la condizione a riga 9? Cioè 9
if (!passed[j]) then goto 3;
Quello che succede è che il j-esimo thread può essere passato ed aver accesso alla Critical Section, e la condizione passed[j]==True, allora la condizione !passed[j] == false, ed il thread i-esimo che ha effettuato questo check procede senza ritornare in cima al ciclo di partenza, ed allora non vi è più mutua esclusione
- Cosa cambia se nella riga 5 si sostituisce :
“if (!interested[k]) then k=i” con “if (interested[k]) then k=i”? c’è solo starvation perché il primo ciclo while, il k-esimo processo passa, ma nel frattempo che arriva al secondo ciclo for, potrebbe passare qualcun altro, facendo sì che il k-esimo ritorni al primo ciclo ed inoltre che k assume un nuovo valore. Inoltre il nuovo k, anche quando ricicla prima di effettuare il check di riga 3, può essere che un altro processo sovrascriva k, rispingendolo dentro al ciclo while. A fortuna un processo potrebbe sperare che esce dal primo ciclo while e nel frattempo i restanti processi rimangono chiusi nel primo ciclo, permettendogli di superare il secondo ciclo for ed entrare nella CS. Questo però è puramente casuale e provoca una forte Starvation.

Baker's Algorithm

Initially

/* global info */

boolean choosing[N] = {false, ..., false};

integer num[N] = {0, ..., 0};

/* local info */

int i = <entity ID>; // $i \in \{0, 1, \dots, N-1\}$

repeat

1 NCS

2 choosing[i] := true

3 num[i] := 1 + max {num[j] : $0 \leq j < N$ }

4 choosing[i] := false

5 for j := 0 to N-1 do begin

6 while choosing[j];

7 while num[j] != 0 AND {num[j], j} < {num[i], i};

8 end

9 CS

10 num[i] := 0

Forever

Il num[i]=0 indica la volontà di non entrare nella critical section.

Possibili domande:

- Cosa cambia se in riga 3 non si somma il massimo dei numeri assegnati ai thread precedenti con l'1? Cioè
num[i] := max {num[j] : $0 \leq j < N$ }
allora tutti avranno il numeretto pari a 0 a prescindere e la condizione num[j] != 0 in riga 7 è sempre false poiché num[j] = 0, ed allora poiché la condizione per rimanere nel ciclo non è soddisfatta, tutti i thread escono immediatamente ed addio mutua esclusione
- Cosa cambia se in riga 10 mettessi num[i] = 1? Allora il primo thread riesce pure a passare, ma si tiene il numero 1 finché non ritorna alla riga 3 a prendersi il numero finale nuovamente. Questo può provocare starvation in quanto finché il processo che è passato non si prende il numerino finale, allora i restanti processi attendono.
- Cosa succede se si inverte la riga 9 con la 10? Allora succede che non esiste più la mutua esclusione in quanto il processo che è uscito dal ciclo, prima di entrare dice che non ha volontà di entrare nella critical section, e quando ci entra prima che finisca permette ai restanti processi di uscire dal ciclo ed entrare insieme a lui nella CS.

